A Derived-Key Security Layer for Opportunistic Bluetooth Mesh Bubbles: Design and Analysis

BUBBLE-BLUE Project — Security Specification

Companion to arXiv:2509.00967

Working draft — July 16, 2026

Abstract

We specify the security layer of Bubble-Blue, a leaderless, infrastructure-free Bluetooth mesh for closed groups ("bubbles") operating over multi-hop opportunistic links. The layer provides group confidentiality, per-packet authenticated encryption, replay protection, private member-to-member channels, and in-band membership revocation, at a fixed 42-byte per-packet overhead. Its central design principle is that *no key material ever travels on data packets*: every packet key is *derived* locally from a shared secret and public header fields via HKDF-SHA256, and pairwise keys are obtained by X25519 ECDH rather than distribution. Revocation and key rollover are unified under a monotone *epoch* mechanism: a coordinator floods a single multi-recipient rekey message (reusing a wrapped-key header adapted from the multi-key scheme of the originating preprint), and stragglers self-heal through epoch-tagged Hello gossip. We describe the packet formats and the encryption/decryption algorithms in full, quantify the overhead, and give a descriptive security analysis against a structured adversary model, together with an explicit list of assumptions and residual limitations.

1 Introduction and Scope

Bubble-Blue targets small closed groups that must communicate over Bluetooth without any fixed infrastructure, where the physical topology is an opportunistic mesh and nodes relay each other's traffic. The origin preprint (arXiv:2509.00967) establishes the operational requirements: a tamper-proof exchange, protection of the control plane against outsiders, a fresh session key per packet, member repudiation, member addition, resilience to loss of the leader, and replay resistance via timestamps.

This document specifies the *security layer as implemented* in Phase 1B (Bluetooth Classic transport, with a BLE profile targeted for Phase 2). It is a companion to the systems preprint and is self-contained: notation and primitives are defined in §2, the key hierarchy in §3, the packet-level AEAD construction in §4, replay handling in §5, the control plane (Hello and the pubkey `ANNOUNCE` beacon) in §6, the group life cycle (epochs, the multi-recipient rekey flood, and gossip catch-up) in §7, and the security analysis with assumptions and limitations in §8–§10.

Relation to the multi-key scheme of the preprint. The preprint proposes a per-packet header carrying the session key wrapped once per recipient (16 bytes $\times\, n$), and repudiation by locally zeroing a recipient slot. We retain the wrapped-key header *idea* but move it from the frequent data path to the rare control path: the header appears only in the revocation message, where its multi-recipient structure is exactly what is needed, while data packets carry a single 16-byte tag. This preserves the four operational properties of the original repudiation (instantaneous, no blocking ceremony, absentee-tolerant, isolation of the revoked member) while eliminating its two defects for our target: an $O(n)$ permanent per-packet cost that breaks the BLE MTU at moderate n, and a purely local revocation state that can silently diverge between senders.

2 Preliminaries and Notation

2.1 Cryptographic primitives

All primitives are standard and instantiated from a single vetted library (`cryptography`, i.e. OpenSSL-backed).

- **AEAD:** AES-256-GCM. We write $c \leftarrow \mathsf{AEAD.Enc}_K(N, m, A)$ for encryption of plaintext m under key K, nonce N, and associated data A, producing ciphertext-with-tag c (the 16-byte tag is appended). Decryption $m \leftarrow \mathsf{AEAD.Dec}_K(N, c, A)$ returns m or the distinguished symbol $\perp$ (`InvalidTag`) on any authentication failure.
- **KDF:** $\mathsf{HKDF}(ikm; salt, info, \ell)$ denotes HKDF-SHA256 producing ℓ bytes from input keying material ikm, with optional salt and info. We fix $\ell = 32$ throughout.
- **Key agreement:** X25519. $\mathsf{ECDH}(sk, pk)$ is the raw Diffie–Hellman shared secret of private key sk and public key pk.
- **MAC/PRF:** HMAC-SHA256, used to derive pseudonymous identifiers.

2.2 Nonce discipline

Every AEAD invocation in the data and control paths uses a *fixed zero nonce* $N = 0^{96}$. This is safe because the encrypting key is itself unique per packet: as shown in §3, packet keys are derived with the tuple (src, seq, ts) mixed into the KDF `info`, so a given key encrypts exactly one message. The (key, nonce) pair is therefore never repeated, which is the sole condition GCM requires. This choice removes 12 bytes of on-wire nonce per packet. The one exception is the Hello private part (§6), whose plaintext contains no sequence number; it uses a fresh random nonce prepended to the ciphertext.

2.3 Wire fields

A *routing header* H of $\rho = 26$ bytes precedes every packet:

$$H = (type,\ src,\ dst,\ ttl,\ seq,\ ts),$$

with src, dst the 6-byte Bluetooth device (BD) addresses, seq a per-source `uint32` counter, and ts a `uint64` millisecond timestamp. We write $\bar{H}$ for the *AAD image* of the header: H with the mutable ttl byte set to zero, so relays may decrement ttl without invalidating the tag (as with mutable fields in IPsec AH).

3 Key Hierarchy

3.1 Bootstrap and identities

A single 32-byte pre-shared group key gk_0 (the bubble secret) is distributed out of band before the mission, as in the preprint's model. All long-term secrets are *derived* from gk_e (the group key of the current epoch e) and public inputs; nothing is transmitted.

Pseudonymous identifier. Each node advertises an 8-byte pseudonym instead of its BD address:

$$aid(gk, bd) = \mathsf{HMAC}_{gk}(bd)\,[0{:}8]. \tag{1}$$

X25519 identity. The recommended (file-less) identity derivation binds the key pair to the group secret and the node address:

$$sk_{bd} = \mathsf{HKDF}\big(gk;\ \text{“bb-identity-v1”},\ \text{“bubble-blue-x25519-identity:”} \,\|\, bd,\ 32\big), \tag{2}$$

$$pk_{bd} = sk_{bd} \cdot G, \tag{3}$$

with G the Curve25519 base point. A file-persisted random identity is also supported (§10); the two strategies trade off stateless operation against intra-group unlinkability of the private key.

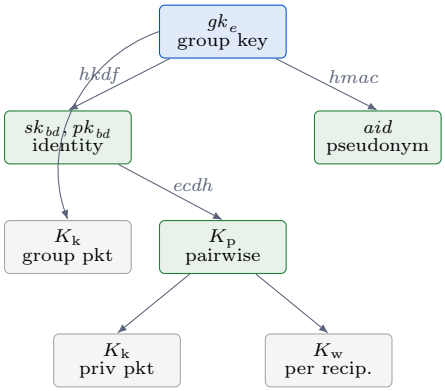


Figure 1: Key hierarchy. Everything descends from the epoch group key gk_e by local derivation; only public keys pk_{bd} ever appear on the wire, and only in the control plane. Group packet keys come straight from gk_e; private packet and wrap keys come from the ECDH pairwise key.

3.2 Pairwise keys

For members A, B that have learned each other's public keys, the pairwise key at epoch e is

$$K_{\mathrm{p}e}^{A,B} = \mathsf{HKDF}\big(\mathsf{ECDH}(sk_A, pk_B);\ \text{“bubble-blue-v1”},\ I_e,\ 32\big), \tag{4}$$

where the domain-separation info is $I_e = \varepsilon$ (empty) for $e = 0$ and $I_e = seq_{\mathrm{u32}}(e)$ otherwise (wire-compatibility for the initial epoch). By symmetry of ECDH, $K_{\mathrm{p}e}^{A,B} = K_{\mathrm{p}e}^{B,A}$; the key is never transmitted, and a third member cannot compute it without one of the two private keys (§8).

3.3 Per-packet keys

Data and unicast control packets derive a fresh AEAD key from a base key $K_{\mathrm{base}} \in \{gk_e, K_{\mathrm{p}}\}$:

$$K_{\mathrm{k}} = \mathsf{HKDF}\big(K_{\mathrm{base}};\ salt{=}\varnothing,\ src \,\|\, seq_{\mathrm{u32}} \,\|\, ts_{\mathrm{u64}},\ 32\big). \tag{5}$$

Uniqueness of (src, seq, ts) gives uniqueness of K_{k}, which underpins the zero-nonce discipline (§2.2).

3.4 Wrap keys

The multi-recipient control messages (subgroup multicast and rekey flood) wrap a one-time message key k_{m} per recipient. The wrap key is derived from the pairwise key with a *distinct* salt, so wrap keys and packet keys are cryptographically separated (a wrap key can never coincide with a payload key, precluding cross-use under the zero nonce):

$$K_{\mathrm{w}} = \mathsf{HKDF}\big(K_{\mathrm{p}};\ \text{“bb-subgroup-wrap-v1”},\ src \,\|\, seq \,\|\, ts,\ 32\big). \tag{6}$$

Figure 1 summarizes the hierarchy.

Algorithm 1 Send data packet

Require: base key K_{base}, plaintext m, source *src*, dest *dst*

$seq \leftarrow seq + 1$; $ts \leftarrow \text{now_ms}()$
$H \leftarrow (type, src, dst, ttl_{\max}, seq, ts)$
$K_{\text{k}} \leftarrow \mathsf{HKDF}(K_{\text{base}};\ src \,\|\, seq \,\|\, ts)$
$c \leftarrow \mathsf{AEAD.Enc}_{K_{\text{k}}}(0^{96},\ m,\ \bar{H})$
return $H \,\|\, c$

Algorithm 2 Receive data packet

Require: packet $P = H \,\|\, c$, candidate base keys $\mathcal{K}$

parse $H = (type, src, dst, ttl, seq, ts)$
if $\neg$ ReplayOK(src, seq, ts) **then return** drop
end if
for $K_{\text{base}} \in \mathcal{K}$ **do** ▷ current then bootstrap
  $K_{\text{k}} \leftarrow \mathsf{HKDF}(K_{\text{base}};\ src \,\|\, seq \,\|\, ts)$
  $m \leftarrow \mathsf{AEAD.Dec}_{K_{\text{k}}}(0^{96},\ c,\ \bar{H})$
  if $m \neq \perp$ **then return** m
  end if
end for
return drop ▷ not a member / tampered

4 Packet-Level Authenticated Encryption

4.1 Data packet format

A data packet is

$$P = H \,\|\, c, \qquad c = \mathsf{AEAD.Enc}_{K_{\text{k}}}(0^{96},\ m,\ \bar{H}),$$

i.e. the 26-byte header in the clear followed by ciphertext-with-tag. The fixed overhead is $\rho + \tau = 26 + 16 = 42$ bytes; with a Bluetooth Classic MTU of 1021 bytes the maximum payload is 979 bytes. Group traffic uses $K_{\text{base}} = gk_e$; private unicast traffic uses $K_{\text{base}} = K_{\text{p}}$.

Algorithms 1 and 2 give the sender and receiver procedures. The receiver reconstructs K_{k} from the cleartext header, so no key or nonce is transmitted.

4.2 Integrity scope

The tag authenticates both c and $\bar{H}$. Any modification of *src*, *dst*, *seq*, *ts*, or *type*, or of the ciphertext, yields $\perp$ and a silent drop. Only *ttl* is malleable, by design, so that relays can forward without membership: a relay reads *dst* and *ttl*, decrements *ttl*, and re-emits the opaque bytes, never deriving a key.

5 Replay Protection

Replay resistance combines three independent mechanisms.

1. **Sliding window.** Per source, a ReplayFilter tracks a window of the last $W = 256$ sequence numbers, tolerating the out-of-order delivery inherent to multi-hop while rejecting any repeat. Two independent filters are maintained — one on the relay path, one on the delivery path — with separate locks, so neither path can mask the other.
2. **Timestamp bound.** A packet whose *ts* deviates from local time by more than $\Delta = 120\,\text{s}$ is rejected before the window is consulted, bounding the delayed-replay window.
3. **Forward dedup.** A bounded cache (TTL 240 s, $\leq$ 4096 entries) keyed by (src, seq) suppresses duplicate relays of a flooded packet.

This is strictly stronger than the "list of the last ten timestamps" of the preprint. Crucially, *seq* and *ts* do double duty: they feed both the replay filter and the key derivation (5). An attacker who alters *seq* to slip past the window simultaneously breaks K_{k} and $\bar{H}$, yielding $\perp$; the two mechanisms interlock.

6 Control Plane

6.1 Two-part Hello

Hellos are emitted every 5 s and are the only messages partly readable by non-members, because the connected-dominating-set (CDS) backbone is built from them. A Hello has a cleartext public part (the emitter pseudonym, its neighbor pseudonyms, and the elected multipoint-relay set) and a private part encrypted under gk_e carrying the true BD addresses, the emitter's X25519 public key, and its current epoch. The public part is the AEAD associated data of the private part, so an outsider can neither read the private fields nor forge a Hello whose public claims would be accepted: tampering with any public field invalidates the private tag. The Hello private part uses a fresh random nonce (no *seq* available for key uniqueness).

6.2 Pubkey diffusion beyond two hops: `ANNOUNCE`

Hellos travel at most two hops, so a member cannot learn the public key of a peer it never hears directly — and a *silent* member (one that never emits data) is never learned at all. Because pairwise keys, subgroup multicast, and the rekey flood all require the recipient's public key, missing keys would silently exclude members from private messaging *and* from revocation.

We therefore add a recurrent, network-wide beacon `PKT_ANNOUNCE` carrying the binding $(bd \rightarrow pk_{bd}, e)$, encrypted under gk_e exactly like a group data packet (Eq. 5, same AEAD path). It is flooded with the existing broadcast/dedup machinery, emitted as a short startup burst

(3× at 2 s spacing), then periodically (30 s), and once immediately after every epoch adoption.

Binding-conflict policy. Because any group member can, in principle, emit an `ANNOUNCE` with a forged *src*, bindings follow a fixed precedence: a key learned from a *direct Hello* private part is authoritative and is never overwritten by an `ANNOUNCE`; absent a Hello, the *first* announced binding is kept; any later conflicting binding is ignored and logged (`[SEC]`). The receiver additionally checks that the announced *bd* equals the authenticated header *src*. This contains, but does not cryptographically prevent, intra-group spoofing; a signed binding is the planned hardening (§10).

Algorithm 3 Coordinator: revoke member M

Require: target M, epoch state (e, gk_e, R_e)
1: **assert** local node is coordinator; ${pk_M}$ known; ${pk_M} \notin R_e$
2: $T \leftarrow$ members $\setminus$ $(\{M\} \cup \{\text{no-pubkey}\})$
3: **for** $j \in T$: ${K_\text{p}}^j \leftarrow$ derive at epoch e ▷ pre-bump
4: $(e', gk', R') \leftarrow (e{+}1,\ \text{rand}_{32},\ R_e \cup \{{pk_M}\})$; persist
5: partition T into MTU-safe subsets $T_1, \ldots, T_p$
6: **for all** T_i **do**
7: build `REKEY_FLOOD` over T_i via Eq. (8)
8: broadcast on all L2CAP links
9: **end for**
10: clear pairwise cache; emit `ANNOUNCE`

7 Group Life Cycle: Epochs, Revocation, Gossip

7.1 Epoch state

Group state is a triple $(e,\ gk_e,\ R_e)$: a monotone epoch counter, the group key of that epoch, and a set R_e of revoked public keys. It is persisted atomically (temp file then rename, mode `0600`, magic+version checked on load), so a rebooted node resumes at its last epoch; a node with lost or corrupt state falls back to $e = 0$ and is treated as an absentee (§7.3). The *coordinator* is elected by convention as the member with the smallest pseudonym among the live members (Eq. 1); there is no static leader to lose, and the convention re-elects a successor with no extra messages.

7.2 Revocation as a single multi-recipient flood

To revoke member M, the coordinator bumps the epoch, samples a fresh group key, adds ${pk_M}$ to the revoked set, and floods a single `PKT_REKEY_FLOOD` whose header wraps a one-time key k_m once per non-revoked recipient. The wire layout reuses the subgroup case header: a count n_cases followed by n_cases cases of $(\text{tag}_8 \,\|\, \text{blob}_{48})$, where the tag is the recipient pseudonym and the blob is k_m wrapped under that recipient's wrap key; the body is the rekey plaintext $(e{+}1 \,\|\, gk_{e+1} \,\|\, R_{e+1})$ encrypted under k_m. Formally, for recipient j with pairwise key ${K_\text{p}}^j$:

$$
\begin{aligned}
{K_\text{w}}^j &= \mathsf{HKDF}({K_\text{p}}^j;\ \text{“bb-subgroup-wrap-v1”},\ \mathit{src} \,\|\, \mathit{seq} \,\|\, \mathit{ts}),\\
\mathit{blob}_j &= \mathsf{AEAD.Enc}_{{K_\text{w}}^j}(0^{96},\ k_\text{m},\ \mathit{aid}_j), \qquad (7)\\
c &= \mathsf{AEAD.Enc}_{k_\text{m}}(0^{96},\ e{+}1 \,\|\, gk_{e+1} \,\|\, R_{e+1},\ A),\\
A &= \bar{H} \,\|\, \langle\text{REKEY_FLOOD}\rangle \,\|\, \text{digest(cases)}. \qquad (8)
\end{aligned}
$$

Two properties follow from the AAD construction. First, the per-recipient AAD aid_j binds each blob to its slot, so a case cannot be replayed into another recipient's position. Second, digest(cases) — a SHA-256 over the ordered case bytes — is inside the body AAD, so tampering with *any* case (including deleting or inserting one) makes the body fail to decrypt for *every* recipient. The absence of the revoked member's case is thereby sealed. A distinct mode marker $\langle$REKEY_FLOOD$\rangle$ separates this construction from the byte-identical subgroup format, so one can never be reinterpreted as the other.

Pre-bump wrap epoch. The wrap keys ${K_\text{p}}^j$ are those of the *current* epoch e, derived *before* the epoch is bumped (the implementation captures e and derives all pairwise keys prior to `prepare_revocation`, then clears the pairwise cache). Recipients unwrap with their own current-epoch pairwise key; a recipient already ahead (via gossip) simply fails to unwrap and is caught up by the unicast path instead.

MTU-safe partition. When the cases do not fit one frame, the recipient set is *partitioned* into independent, self-contained packets — each with its own *seq*, *ts*, k_m, and body — rather than wire-fragmented. The budget per packet is

$$
n_\text{max} = \left\lfloor \frac{\text{MTU} - \rho - 1 - |c| - \tau}{56} \right\rfloor,
$$

with 56 the case size. For a single revocation the body is 70 bytes; on a conservative BLE MTU this yields $n_\text{max} = 2$ cases per packet, so a 32-member bubble revokes in $\lceil 31/2 \rceil = 16$ flood packets in the worst BLE case and a single packet on Bluetooth Classic. There is no reassembly: partitions are disjoint recipient subsets.

Algorithm 3 states the coordinator procedure.

7.3 Adoption and gossip catch-up

A recipient locates its own case by pseudonym, unwraps k_m, decrypts the body, and *adopts* the new state iff the received epoch is strictly greater than its own; strict monotonicity rejects replays of stale floods. Absent own case, or on any $\perp$, the node stays silent — the normal outcome for the revoked member and for members carried in a different partition.

The single flood need not reach everyone. Because each Hello private part carries the emitter's epoch, a node that is up to date and sees a Hello with a *stale, non-revoked* epoch replies with the current state over the existing *unicast* `PKT_REKEY` gossip channel (unchanged from the baseline). Since the rekey body carries the *complete* state (e', gk', R'), one catch-up suffices no matter how many revocations were missed. Convergence is one Hello cycle ($\approx 5\,\mathrm{s}$). A revoked member is never caught up: its public key is in R', so no member gossips to it.

The same flood-plus-gossip machinery serves member *addition* and periodic *key rollover*: all three are an epoch bump with a new key, differing only in whether R grows. For urgent traffic during the brief propagation window, an optional `PKT_SUBGROUP` à-la-carte multicast ($O(|S|)$ wrapped cases) carries a message to a named subset immediately.

8 Security Analysis

8.1 Goals

(G1) Group confidentiality: only current, non-revoked members recover group plaintext. **(G2) Integrity/authenticity:** any modification of a packet (payload or immutable header fields) is detected. **(G3) Replay resistance:** a captured packet cannot be re-injected and re-processed. **(G4) Private confidentiality:** member-to-member traffic is readable only by the two endpoints (under the identity assumption of §10). **(G5) Revocation:** after revocation of M, no key held by M decrypts subsequent traffic, globally and consistently.

8.2 Why a third member cannot forge a pairwise key

Goal G4 rests on the computational Diffie–Hellman assumption on Curve25519. A member C observing $A{\to}B$ private traffic knows $pk_A = sk_A \cdot G$ and $pk_B = sk_B \cdot G$ and must obtain $K_{\mathrm{p}}{}^{A,B} = \mathsf{HKDF}(sk_A sk_B \cdot G; \dots)$. The only group operation on points is addition, and $sk_A \cdot G + sk_B \cdot G = (sk_A + sk_B) \cdot G \neq sk_A sk_B \cdot G$; recovering $sk_A sk_B \cdot G$ from the two public points is the CDH problem. Hence two public keys never suffice; one private key is required. (This holds against outsiders unconditionally; the intra-group caveat is the derived-identity assumption of §10.)

8.3 Adversary model and coverage

Table 1 states, for each adversary capability, the mechanism that defends against it. We consider outsiders (no group key), active network attackers, replay, a detected compromised insider, a stolen device, and an external relay.

8.4 On the zero nonce

The zero-nonce choice (§2.2) is safe precisely because Eq. (5) makes each key single-use: the (key, nonce) pair is unique, which is GCM's only requirement. Including ts in the derivation additionally prevents a reused key after a counter reset on reboot. Wrap keys and packet keys are salt-separated (Eqs. 5, 6), so the same AES key is never used with two different messages under the zero nonce.

9 Overhead

Table 2 compares the on-wire cost with the multi-key scheme of the preprint at $n = 32$.

10 Assumptions and Limitations

We state these explicitly; none is a regression relative to the preprint, and two pre-existed there in a worse form (no detection, no repair).

1. **Unsigned rekey (primary).** A malicious but legitimate member could emit a rekey and attempt to fork the group. Present mitigations: floods are accepted only through the authenticated pairwise wrap, epochs are strictly monotone, and emission is restricted to the coordinator by convention (non-coordinators raise an error). The missing piece is *receiver-side* verification that the emitter is the coordinator; a signed rekey is the next increment. The preprint likewise authenticates repudiation by nothing.

2. **Derived-identity trust anchor.** With the recommended (file-less) identities, $sk_{bd} = \mathsf{HKDF}(gk; \dots)$: any holder of the group key can re-derive any member's private key and thus any K_{p}. Private confidentiality (G4) therefore holds against outsiders, not against a motivated insider, under this strategy. The file-persisted random identity (`_load_or_create_file_private_key`, mode `0600`) restores the strong intra-group guarantee and is a configuration change, not a code change. This matches the preprint's short-mission, initially-trusted model.

3. **Propagation window.** During the seconds between decision and convergence, a not-yet-switched member may emit under the old key, readable by M. `PKT_SUBGROUP` mitigates for urgent traffic. The preprint's local repudiation had a permanent, undetectable analogue.

4. **No intra-epoch forward secrecy.** Compromise of gk_e exposes epoch-e traffic. Consistent with the preprint (keys not renewed for the mission); epoch rotation already bounds exposure, and periodic rollover is the same mechanism at no extra cost.

Adversary / capability	What they can attempt	Defending mechanism (goal)
Outsider, passive	Observe all frames	Payloads and Hello private parts are AES-GCM under gk_e; only pseudonyms and CDS structure are visible; BD addresses and public keys never appear in cleartext (G1).
Outsider, active	Inject or modify frames	Tag over c and $\bar{H} \Rightarrow \perp$; forged Hello rejected because the public part is the private part's AAD (G2).
Any party	Replay a captured frame	Dual sliding window (W=256) + timestamp bound (Δ=120 s) + relay dedup; *seq*, *ts* also bind K_{k} (G3).
Compromised insider (detected)	Keep using held keys after detection	Epoch bump: gk_e and all K_{p} of M stop decrypting; $pk_M \in R$ blocks any future catch-up (G5).
Stolen device	Same as above, no detection latency assumed	Identical mechanism; revocation is in-band, one tool for both threats (G5).
External relay (nonmember)	Drop, delay, or duplicate	Cannot read (no gk_e) or modify (tag); duplication caught by dedup; worst case is local denial, inherent to wireless.

Table 1: Adversary model and defensive coverage. Goals G1–G5 as in §8.

	Preprint	This work
Per-packet key header	$16n = 512$ B	0 B
Fixed per-packet overhead	$\rho + 16n$	$\rho + \tau = 42$ B
Typical chat packet	~ 600 B	~ 88 B
Fits BLE MTU (251 B)	no	yes
Initial keying	$O(n^2)$ out of band	1 PSK + ECDH
Add a member	n keys out of band	1 flood (e+1)
Revoke a member	local slot zeroing	$\lceil (n-1)/n_{\max} \rceil$ floods

Table 2: Overhead comparison at $n = 32$. The wrapped-key header is confined to the rare rekey flood rather than every data packet.

5. **Not fully stateless.** Epoch, key, and revoked set are persisted (~ 100 B, atomic, `0600`); an amnesiac node is caught up like an absentee.

6. **Rekey-flood metadata.** The case count and recipient pseudonyms are cleartext, so an observer learns that a rekey occurred and estimates group size; no padding or decoy cases are added. The prior $n-1$ unicasts leaked less.

7. **Pseudonym recomputation coupling.** Reaching a member known only via `ANNOUNCE` recomputes its pseudonym from the bootstrap PSK (Eq. 1); this is sound only while all nodes share that PSK as the pseudonym key. A future per-epoch pseudonym would require carrying the pseudonym in `ANNOUNCE` instead.

8. **Collusion.** A member that relays the current key to M defeats any revocation; this is irreducible in any shared-key scheme, including the preprint's.

11 Conclusion

The Bubble-Blue security layer meets the operational requirements of the originating preprint at a fixed 42-byte per-packet cost, by replacing key *distribution* with key *derivation* throughout, and by confining the multi-recipient wrapped-key header to a rare, self-healing epoch flood rather than every data packet. The construction is implemented and unit-tested, including the adversarial cases of Table 1 (outsider decryption failure, tamper detection, replay rejection, revoked-member exclusion, two-hop adoption, and gossip catch-up). The most consequential hardening — a signed rekey with receiver-side coordinator verification — is identified as the next increment and does not alter the wire formats specified here.